TalonOS

Technical Reference & Developer SDK Guide

Architecture · hardware · animator CPU · shader GPU · network · TalonSync link · SDK

GPU TalonOS Core shader · 3 programs

CPU FX animator · 18 layers

MEM registers + saved parameters

NET 10 synced bits · TP/Net channel

I/O contacts · stylus · OSC bus



FOR DEVELOPERS AND AVATAR CREATORS

Talon Pilot 2.23.1 · TalonOS Nightwing · TalonSync 1.14.4 · Link protocol 3 · Created by NevvyDrone

Contents

Contents 2

Introduction 4

- Audience and scope . . . 4
- Conventions . . . 4

System Architecture 5

- Design principles . . . 5

Hardware Reference 7

- Physical model . . . 7
- Input contacts . . . 7
- Constraints, PhysBone and audio . . . 8

CPU: The Animator 9

- Parking . . . 9
- Input dispatch . . . 9
- Power, idle and desktop state machines . . . 10

Memory: Parameters and Registers 11

- Parameter budget . . . 11
- Text slots . . . 11
- The register file . . . 12

GPU: The TalonOS Core Shader 14

- Programs and screen ownership . . . 14
- Modules . . . 15
- The app contract . . . 16
- Shader coding rules . . . 16
- ROM map . . . 16

Network: What Other Players See 17

- Beam: data over contacts . . . 17

The TalonSync Link Protocol 19

- Signals . . . 19
- Address map . . . 20
- A TalonSync session . . . 21

TalonSync Software Architecture 22

- Scripting the PDA from Python . . . 23

Developer SDK Guide 24

- The toolchain . . . 24

Recipe: add a preference . . . 24
Recipe: add a compiled-in string . . . 24
Recipe: add an app . . . 25
Optional apps (2.14) . . . 25
Power-on self-test (2.17) . . . 26
Terminal: the PC as the CPU (2.18) . . . 26
Clocks without layers (2.15) . . . 27
Recipe: add a bus command . . . 28
Compatibility rules . . . 29
Performance budget . . . 29
Testing checklist . . . 29

Appendices **30**

A. Package layout . . . 30
B. Glossary . . . 30

01

Introduction

How Talon Pilot is built, how its parts talk to each other, and how to extend it.

Talon Pilot is a handheld computer that exists entirely inside a VRChat avatar. It has no scripts: VRChat avatars cannot run code. Instead, TalonOS maps a classic computer architecture onto the pieces an avatar is allowed to have. This document describes each of those pieces, the contracts between them, and the developer kit for building on them.

Computer part	Talon Pilot implementation	Where
Processor (CPU)	The FX animator controller: state machines whose transitions are the instructions and whose parameter drivers are the writes.	`Editor/TalonPilotAnimator*.cs`
Registers / RAM	Material properties on the screen renderer (the register file), plus animator parameters as working memory.	`Shaders/Core/Talon_Registers.cginc`
Storage	Saved local expression parameters: typed text, preferences, pet, app settings. They survive avatar reloads.	`TalonPilotWizard.cs (EPar)`
GPU / display	The TalonOS Core shader draws the whole 320 × 480 screen every frame from the registers.	`Shaders/Core/*.cginc`
ROM	A 32 × 1024 texture: one ASCII line per row (app titles, built-in text, prefs labels, keyboard).	`Editor/TalonPilotRom.cs`
Input devices	Contact receivers (touch grid, ink grid, buttons), the stylus senders, Expressions menu, VRChat built-ins.	`Wizard install + Mesh`
Network interface	Synced parameters: TP/On, TP/Net (a time-multiplexed 8-bit channel), TP/Beam and TP/JazzAll.	`TalonPilotAnimatorNet.cs`
Serial port	The TalonSync link: a clocked, acknowledged bus over OSC parameters.	`TalonPilotAnimatorBus.cs`, `talonsync/bus.py`

Audience and scope

- **Avatar creators** who want to understand what Talon Pilot adds to their avatar and what it costs.
- **Developers** who want to add apps, preferences, strings or bus commands to TalonOS, or script TalonSync.
- Everything here describes the shipping source: Unity package `Assets/TalonPilot` and the TalonSync Python application.

Conventions

- Parameters are written `TP/App`; material registers `_App`; files `Talon_Apps.cginc`.
- Virtual screen coordinates: x 0..320 left to right, y 0..480 top to bottom. The content area of an app is y 28..444.
- Rules marked **RULE** are hard constraints. Breaking them causes compile failures, lag or corrupted saved data.

02

System Architecture

One avatar, one computer, one OSC link.

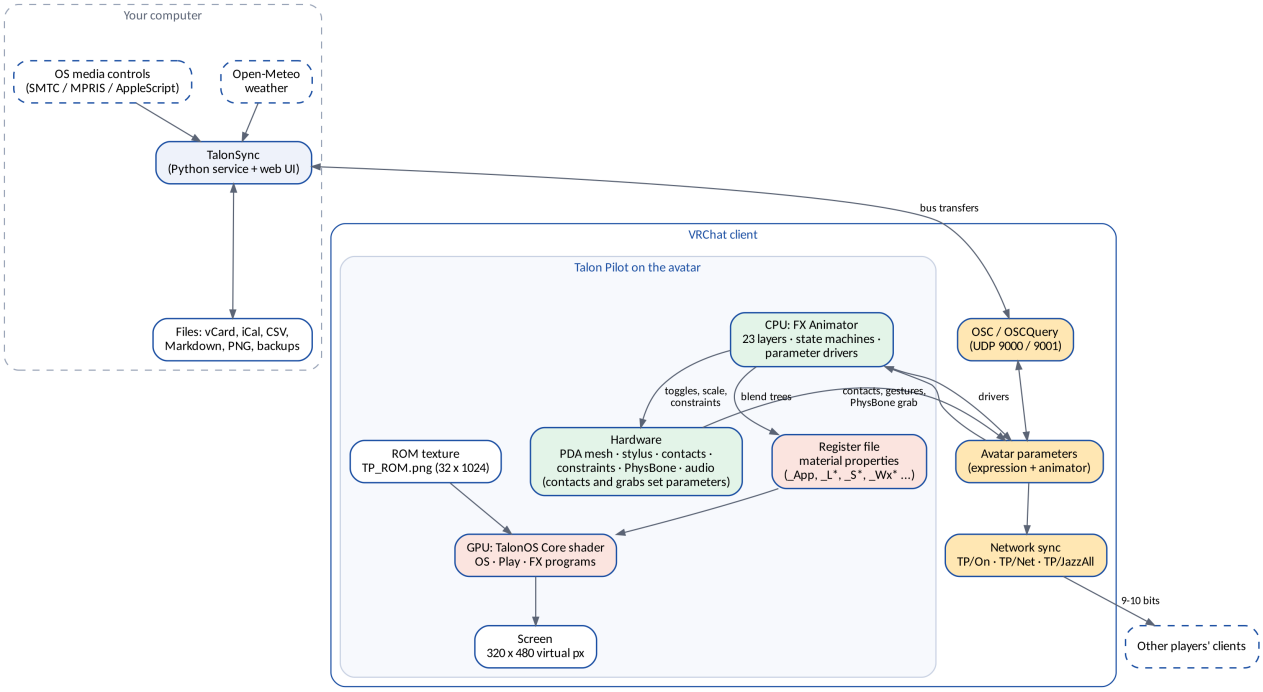


Figure 2-1. System overview: the avatar's components, VRChat, TalonSync and other players.

Every frame, inputs arrive as animator parameters (contacts, menu toggles, OSC writes from TalonSync, VRChat built-ins such as `VRMode` and `Voice`). Animator layers react by changing state; states run parameter drivers that update working parameters. Direct blend trees copy those parameters into material properties on the screen, and the TalonOS Core shader turns the material properties into pixels. Three parameters are synced so other players' copies of the same animator can show your app, theme, size and weather.

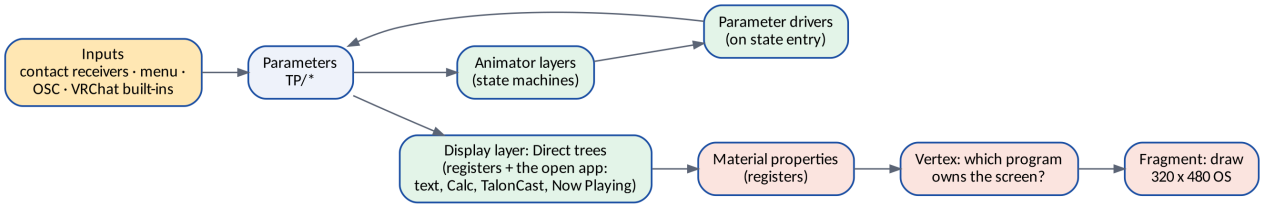


Figure 2-2. One frame, input to pixels.

Design principles

- **Nothing runs that isn't visible.** Layers park when the PDA is off or on other players' clients; blend trees only animate the app on screen; the shader only runs the program that owns the screen.
- **Local first, sync little.** All of your information lives in unsynced, saved parameters. Only 10 bits (11 with shared jazz) cross the network.

- **Generated, not hand-written.** The animator (about 2,200 states) is built by C# generators; shader tables, registers and strings by Python generators. Edit the generators, then regenerate.
- **Fail safe.** An unknown app shows an error dialog inside the OS; missing optional SDK features fall back (for example, desktop parking without VRC constraints).

03

Hardware Reference

The physical device: objects, colliders and constraints the wizard builds.

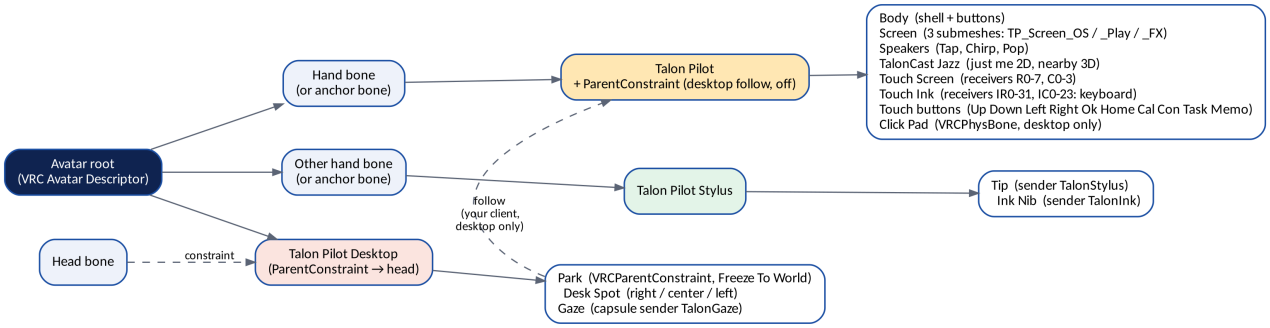


Figure 3-1. GameObject hierarchy created by the Install Wizard.

Physical model

Part	Size (1× scale, metres)	Notes
Body	0.078 × 0.121 × 0.015 (Classic)	Lies flat in its local space: X width, Z length (top at +Z), screen faces +Y. 2.13 case styles: Slim Metal 78×121×11, Rugged Field 86×131×19 (+ bumpers), Retro Brick 84×134×24, Clear Plastic = Classic. The top face, screen and buttons are identical on all of them; submesh 0 shell, 1 accents. UV0 is a box projection for the 2.14 finishes (8 cm per repeat, running along the length); UV1 carries the logo print on the back (neutral grey everywhere else), used by the Standard shader's Detail Albedo x2.
Screen	0.058 × 0.087 (2:3)	X −0.029..0.029, Z −0.0325..0.0545. UV (0,0) bottom-left; one quad, three submeshes.
App buttons	X ±0.0295, ±0.0175	On the button row at Z −0.046, with the 5-way navigator between them.
Stylus	Length 0.125	Tip at +Z. Tip sender radius 3 mm; Ink Nib radius 0.7 mm. 2.14: a custom model is fitted by TalonPilotMesh.FitStylus(): longest side -> +Z, scaled to 0.125, the narrower end at the tip, centred on the axis (a handedness-preserving axis mapping, so faces keep their winding); the Stylus object, Tip and Ink Nib never move.
Wizard size	0.75 to 3 × (default 1.5 ×)	Then 60 to 160 % in game (Prefs › Size & Power); contacts scale with it.

Input contacts

All receivers are local-only (`localOnly = true`, `allowSelf = true`, `allowOthers = false`), Constant type, and use Box shapes where the SDK supports them (capsules otherwise). Touch slabs reach 2 mm above the glass and 6 mm below it, so pushing into the screen still counts.

Receiver set	Parameters	Listens to	Purpose
Touch Screen (12)	`TP/In/R0..R7, TP/In/C0..C3`	TalonInk (+ fingers, TalonGaze)	Coarse grid: rows of 50 px from y 36, columns of 80 px.
Touch Ink (56)	`TP/In/IR0..IR31, TP/In/IC0..IC23`	TalonInk (+ fingers, TalonGaze)	Fine grid over x 8..312, y 40..440: the keyboard.

Receiver set	Parameters	Listens to	Purpose
Buttons (10)	`TP/In/Up, Down, Left, Right, Ok, Home, Cal, Con, Task, Memo`	TalonStylus (+ fingers, TalonGaze)	Hardware buttons (smaller and lower since 2.22, so they are harder to bump); Home is the owl Home button in the middle of the bottom bar.

A tap is a row AND a column at the same time. Physical buttons listen to the wide stylus tip; the screen listens to the fine nib, which makes taps pinpoint accurate.

Constraints, PhysBone and audio

Component	On	Behaviour
ParentConstraint → head	Talon Pilot Desktop	Static: holds the desktop anchor at the head (never parented, because VRChat shrinks your own head locally).
VRCParentConstraint (no sources)	Park	Animated Freeze To World parks the desktop spot in the world. Added by reflection; skipped on SDKs without VRC constraints.
ParentConstraint (disabled)	Talon Pilot	Source: Desk Spot. Enabled only on your client in desktop mode, so the real PDA moves in front of you.
VRCPhysBone	Click Pad	Immobile, grabbable, parameter TP/Desk/Click → _IsGrabbed is the desktop mouse click. Active only in desktop mode.
AudioSource ×3	Speakers	Tap, Chirp, Pop: 3D, toggled by the Beep and Power layers.
AudioSource ×2	TalonStar Jazz (its object keeps the old name)	2D 'just me' (only your client turns it on) and 3D 'nearby' (others' clients, when TP/JazzAll).

> **NOTE**
> PhysBones on your avatar that contain the PDA or stylus (for example a wing chain) get them added to their Ignore Transforms automatically, so the PDA doesn't swing with the bones.

04

CPU: The Animator

18 layers in your FX controller, generated by TalonPilotAnimator.

Talon Pilot's layers are prefixed `TP` and appended to your FX controller (backed up first). Every layer except the screen trees uses Write Defaults OFF with an empty 1/60 s clip; the register trees use Write Defaults ON so registers fall back to their material defaults.

Layer	Role	Others' clients	PDA off
Display (+ app data)	The always-on register tree, plus in each state the open app's text (2 chars / register), Calc, Now Playing or TalonCast tree. Off: nothing evaluated.	Registers only (TalonCast: shared tree)	Paused
Net Receive / App / Net Send	Decode TP/Net · map TP/App → _App · encode TP/Net	Runs / runs / idle	Runs / runs / parked
Audio · Power · Beep	Speakers, power states, tap beeps	Run / run / parked	Run / run / parked
Sync Screen · TalonSync Bus	TalonSync handshake and the OSC bus	Parked	Run (PC can wake)
Input	All button, tap and key handling (~900 states)	Parked	Parked
Idle · Stopwatch	Screensaver / auto-off timers · stopwatch	Parked	Parked / runs
Pet Life · Pet Anim	Hoot Pet needs and animations	Parked	Run
Size	PDA + stylus scale (VR) or desktop scale	Runs	Runs
Desktop ×2	Desktop (mode + gaze press), Desktop Park (the spot is in Size). The optional cursor joystick adds Cursor, Tap, Rows, Cols	Parked	Parked
TalonSync (optional)	One-time data load after Apply to avatar sends your data	—	—

Parking

Parking adds an AnyState transition (checked first) to a parked state with a single way back. Remote parking uses `IfNot IsLocal`; power parking adds `IfNot TP/On`. Measured per-frame transition checks: about 215 with the PDA on, 120 off, 115 for other players.

Input dispatch

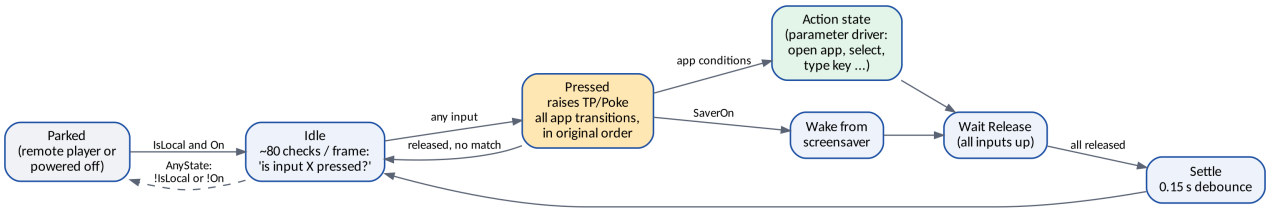


Figure 4-1. Input layer: the Pressed hub keeps idle checks to one per input.

Generators call `On(button, action, conds)`, `Tap(row, col, action, conds)`, `TapRow`, `TapAny` and `Stepper(...)`. Each creates a transition from Idle; `HubInput()` then moves every transition that needs a held input into the Pressed hub, preserving order, and gives Idle one `If TP/In/x` transition per input. Actions are shared by name

(`Action(name, drivers)`), exit to Wait Release and debounce for 0.15 s.

> **RULE**
> Unity compares Int parameters against whole-number thresholds. `FixIntConditions` rewrites every Int Greater/Less so that 'Less 0.5' means == 0 and 'Greater 0.5' means ≥ 1, exactly as in VRChat.

Power, idle and desktop state machines

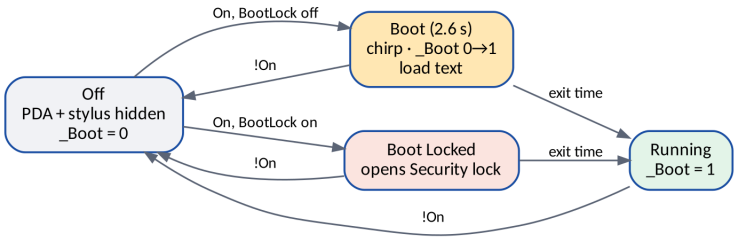


Figure 4-2. Power layer.

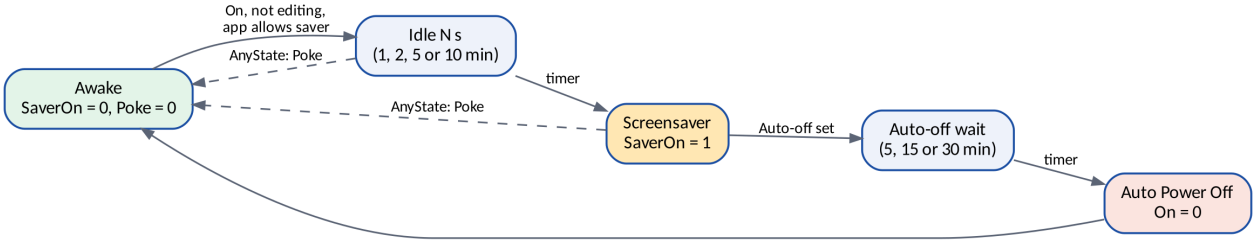


Figure 4-3. Idle layer: screensaver and auto-off. Any input raises TP/Poke.

Where the last press was (2.23). TP/Poke now leads to a *Pressed* state that writes `_TapY` = -200 (no ripple), then to *Pressed row r* if ink row r is held (writing its centre to `_TapY`), then to *Pressed column k* (`_TapX`), then back to Awake: a few frames once per press, and no transitions to check while nobody touches the PDA. The layer is write-defaults off, so the values stay. Awake and the four *Idle n s* states play clips that ramp `_TapT` from 0 at one per second, so the shader knows how long ago the last press was.

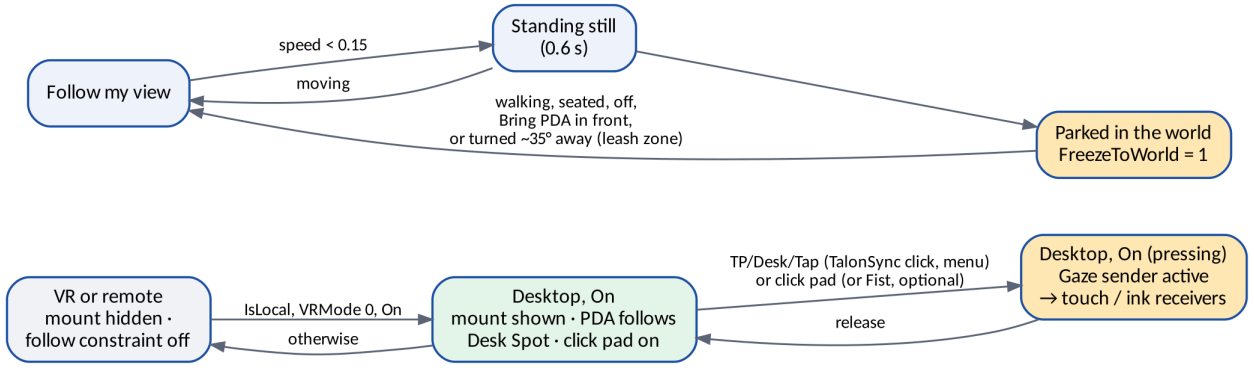


Figure 4-4. Desktop mode: the Desktop layer (mode and gaze press) and Desktop Park.

05

Memory: Parameters and Registers

What is stored, what is synced, and how values reach the screen.

Parameter budget

Group	Type	Count	Synced	Saved
`TP/On`	Bool	1	yes (1 bit)	yes
`TP/Net`	Int	1	yes (8 bits)	no
`TP/JazzAll`	Bool	1	yes (1 bit, if jazz is installed)	no
`TP/Beam`	Bool	1	yes (1 bit): "I'm beaming"	no
`TP/App, TP/Bright, TP/Theme, TP/Sound, TP/Hour, TP/Min`	Int / Float	6	no	yes (App no)
`TP/S/* preferences`	Int	17	no	yes
`Text: TP/Tx/s_i (48 slots × 15) and TP/XL/s`	Float / Int	768	no	yes
`Tasks TP/T0-7, app settings, pet`	Float / Int	~19	no	yes
`TalonSync link (TP/Bus/*, TP/Sync/*)`	mixed	~48	no	no
`Desktop (TP/Desk/Tap, Here; with the cursor option also X, Y, Aim, CTap)`	Bool / Float	2 (6)	no	no
`Animator-only working parameters`	mixed	~270	—	—

Text slots

Text is stored as 48 saved slots of 15 characters, right-aligned (the keyboard shifts characters in from the right). Each character is a saved Float holding `code / 127`, which survives VRChat's float storage exactly.

Slots	Owner	Display lines
0-7	Tasks	0-7
8-22	Memos (5 memos × 3 lines)	8-22
23	Hypno caption	24
24-35	Calendar (6 events × 2 lines)	25-36
36-47	Contacts (6 × 2)	37-48
—	Keyboard edit buffer	23
—	Now Playing (TalonSync)	49-50
—	TalonCast city, ticker	51, 52-54

Display encoding: two characters per register

```
_L{line}_{k} = char(2k) * 128 + char(2k+1)          k = 0..7   (k = 7 holds char 14 only)
max value 127 * 128 + 127 = 16383  ->  float32 holds it to ~0.001

decode (Talon_LCodeOf):
    v  = register k = floor(i / 2)
```

```
hi = floor((v + 0.5) / 128)
code = (i even) ? hi : round(v - hi * 128)
```

Only one app is on screen at a time, so the 55 display lines share a 16-line window of registers (`Talon_LineWindow` in the shader, `LineWindow` in C#): each app's lines start at window line 0, and the keyboard's edit line is window line 15. That is 128 registers instead of 440.

> **RULE**
> Never pack three characters per register. Values up to two million amplify float rounding in saved weights enough to change letters ('Good' became 'food' in 2.5.2).

The register file

Registers	Meaning
`_App, _Boot, _Theme, _Bright, _Sound, _HourOff, _MinOff`	System state
`_Cursor, _Sel, _View, _Edit, _EPage, _PCat`	App navigation
`_S0 .. _S22`	Preference display copies (float mirrors of TP/S/*)
`_T0 .. _T7`	Task done flags
`_CalcKey, _CalcA, _CalcK, _CalcT, _CalcPh, _CalcB0-7, _CalcOp0-7`	Calculator
`_L0_0 .. _L15_7`	Text: a 16-line screen window (see above)
`_Wx*, _WxF0-9, _WxPage, _WxJazz`	TalonCast
`_Sync, _SyncStage, _NPPos`	TalonSync screen, Now Playing position
`_SaverOn, _SW, _HypSpeed, _HypStyle, _ScopeMode, _Voice, _Speed, _3D*`	Apps and screensaver
`_Orb, _SkyPage, _SkyHr, _SkyLat, _SkyLon, _PC0 .. _PC6, _PCPage, _BeamTx`	Owl Oracle, Moon & Stars, PC Link (two values per register: _PC{k/2} = v(2k)*256 + v(2k+1), raw 0-255, 255 = not available; value 5, the low half of _PC2, also drives the title-bar battery), Beam
`_Pet*`	Hoot Pet
`_TouchX, _TouchY, _TouchP, _TouchHover`	Desktop cursor (optional)
`_TapX, _TapY, _TapT`	2.23: the last screen press (centre of its ink-grid cell, px; _TapY < 0 = a button) and the seconds since the last press, set by the Idle layer. Touch ripples and the idle glow
`_Foil`	2.23: Holo foil (from the synced TP/Foil): the launcher icons' foil
`_WallTex, _WallPic, _WallDim`	Wallpaper picture (2.13, static, set by the wizard)
`_CTheme{0-2}{Bg,Fg,Ac,Ai,Fc}, _CThemeInv`	Custom themes 5-7 (2.13, static material colours; declared in Talon_Common.cginc)
`_Launch0, _Launch1, _Launch2`	The launcher table (2.14, static): item 0-31 -> app id, 3 items per float, 6 bits each, stored as id + 3 (0 Back, 1 More, 2 empty). Written by the wizard from TalonPilotAnimator.LauncherFloats(); the default is every app
`_SBState, _SBT, _SBPhase, _SBS0-2, _SBA0-2, _SBScore, _SBLives, _SBBest, _SBPop, _SBPopT`	TalonPop (2.15.2): round state, round ticks (0.4 s) and tick phase, each star's slot and age in ticks, score, lives, best, last pop
`_GlobeT, _GlobeScene`	Snow Globe (2.15): seconds since the last shake + 1 (0 = settled), scene 0 day / 1 dusk / 2 night

Registers	Meaning
`_BootRes`	Power-on self-test report (2.17): (165 + 256 x flags) / 65536, only while booting
`_AquaT, _AquaX, _AquaLight`	Aquarium (2.16): seconds since feeding + 1 (0 = never), feeding column 0-3, lights 0 day / 1 sunset / 2 night / 3 blacklight; 4-7 the same in the koi tank (2.21)
`_AppsOff`	Apps left out (2.14; 2.16 adds 14-19): bit (id - 20) for apps 20-31, bit (id - 2) for apps 14-19
`_AppsOff`	Apps left out (2.14, static): bit (id - 20). Talon_AppInstalled() treats them as not installed, so a stray open shows the error dialog
`_SysError, _ScrimAlpha`	Error handler

Register reads by index go through generated binary-search functions (`Talon_S(i)`, `Talon_Task(i)`, `Talon_GetLine(i)` ...). Constant indices fold away at compile time.

06

GPU: The TalonOS Core Shader

One shader, four programs, one 320 × 480 screen.

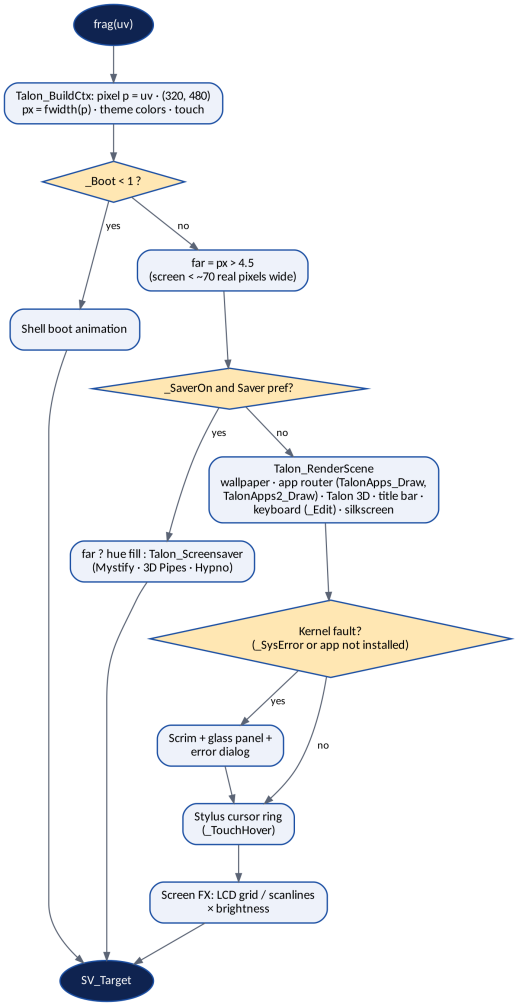


Figure 6-1. Fragment pipeline.

Programs and screen ownership

The screen mesh has four identical submeshes with four materials: `TP_Screen_OS`, `TP_Screen_Play`, `TP_Screen_FX` and `TP_Screen_Aqua` (2.21). Each enables one local keyword (`TALON_PART_OS`, `TALON_PART_PLAY`, `TALON_PART_FX`, `TALON_PART_AQUA`) so it compiles only its own apps. The vertex shader of the programs that don't own the current screen moves its quad out of clip space, so only one program shades pixels.

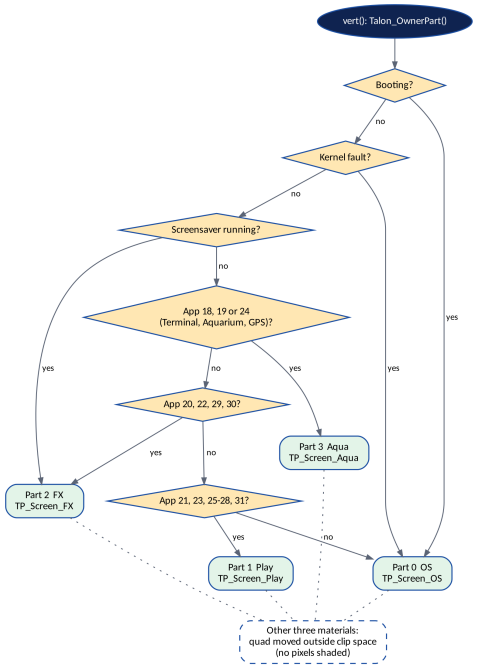


Figure 6-2. Talon_OwnerPart(): which program draws.

Program	Apps	DXIL instructions
OS	Power-on self-test, launcher (0) and its wallpapers, Calendar 1, Contacts 2, Tasks 3, Memos 4, Calc 5, Clock 6, Prefs 7, TalonSync 8, Expense 10, Card Info 11, Security 12, error dialog, keyboard	≈ 18,300
Play	TalonPop 21, Hypno 23, Scope 25 (nine styles), Hoot Pet 26, TalonCast 27, Owl Oracle 28, Beam 31, keyboard	≈ 18,200
FX	Talon 3D 20, Snow Globe 22, Moon & Stars 29 (the 3D moon, 2.21), PC Link 30, screensavers (Mystify, 3D Pipes, Hypnospiral)	≈ 19,600
Aqua (2.21)	Terminal 18 (moved from OS in 2.21), Aquarium 19 (reef and koi tanks), Compass 24 (the glass-cockpit GPS), keyboard	≈ 20,100
(no keyword)	A stub that asks you to re-run the Install Wizard (2.23.1: the wizard's template material and the Core Preview use the OS keyword, so Unity compiles the OS program once). TALON_PART_ALL builds everything for preview tools.	≈ 90

Modules

File	Contents
`TalonOS_Core.shader`	Generated from the template: properties, vertex ownership, kernel router, fragment pipeline.
`Talon_Common.cginc`	TalonCtx, SDF primitives, coverage helpers (T_Rect, T_RRect, T_Seg ...), program-part macros.
`Talon_Registers.cginc`	Generated register declarations and index-search functions.
`Talon_FntMgr.cginc`	ROM text, glyphs, numbers, user lines (TalonLine), icon atlas.
`Talon_Strings.cginc`	Generated compiled-in strings (TALON_S_*).
`Talon_Tables.cginc`	Generated prefs metadata and the keyboard layout.
`Talon_Shell.cginc`	Themes, title bar, keyboard overlay, silkscreen, boot, screen FX.

File	Contents
`Talon_Apps.cginc / Talon_Apps2.cginc`	Built-in apps and the extension pack, with their routers.
`Talon_Star.cginc · Talon_Pet.cginc`	TalonCast and Hoot Pet.
`Talon_Screensaver.cginc · Talon_3DMgr.cginc`	Screensavers; SDF raymarcher and wireframe renderer.
`Talon_CompMgr.cginc · Talon_Time.cginc`	Panels, shadows and glass; VRChat time decoding (UTC, local offset, dates).

The app contract

```
float3 TalonApp_MyApp(TalonCtx c, float3 col)
{
    // c.p   : this pixel in virtual px (x 0..320, y 0..480)
    // c.px  : one real pixel in virtual px (use for anti-aliasing: Talon_Fill(c, d))
    // c.bg, c.fg, c.accent, c.face, c.hilite ... : theme colors
    // draw only the content area, y 28..444; the shell draws the title bar and silkscreen
    col = lerp(col, c.fg, Talon_StrCentered(c, float2(160, 200), TALON_S_MYAPP_HELLO, 1.0));
    return col;
}
```

Shader coding rules

- **No dynamically indexed arrays of runtime values.** They become per-pixel scratch memory. Use generated binary-search functions or selects. Static const tables are fine (they become immediate constant buffers).
- **Draw each primitive once per pixel (the slots pattern).** Decide which string, number or icon this pixel can belong to, then call the drawing routine once. FXC inlines every call site.
- **Fetch a text line once** (Talon_GetLine) and pass the TalonLine around.
- **Derivatives only in uniform control flow**: use c.px, computed once in Talon_BuildCtx.
- **No HLSL reserved words as identifiers**: pass, line, point, sample, shared, groupshared, texture, compile ...
- **Branch on registers with Is(v, n)**, which tests 'equals small integer' with a ±0.5 tolerance.

ROM map

Rows	Contents
`0-31`	App titles (ROW_TITLE)
`32 + (app−1)·64`	Per-app blocks: built-in records, help text, stage names
`900-929`	Theme names, game text, TalonSync messages, keyboard words
`930-937`	Prefs categories
`940-959, 1017-1019`	Pref labels (index < 20, then 20+)
`960-999`	Pref value labels
`1000-1009`	Keyboard rows and key captions
`1010-1016`	About text

07

Network: What Other Players See

Ten bits, multiplexed.

VRChat syncs Bools as 1 bit and Ints/Floats as 8 bits. Talon Pilot syncs `TP/On`, `TP/Net`, `TP/Beam` and optionally `TP/JazzAll`. `TP/App` itself is local; the Net Send layer takes turns writing typed messages into `TP/Net`, and every other client latches each type into its own parameter.

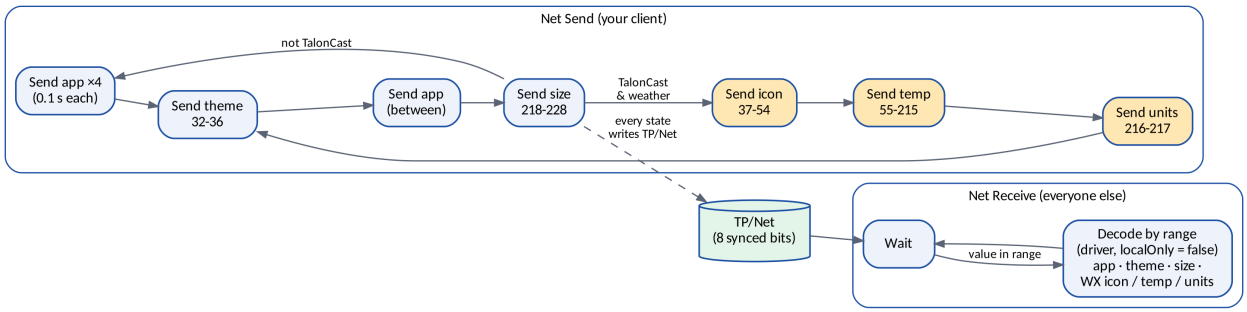


Figure 7-1. TP/Net send loop and receive decoder.

TP/Net value	Message	Decoded into
`0 - 31`	Open app	`TP/App`
`32 - 36`	Theme 0-4	`TP/SF/Theme`
`37 - 54`	TalonCast icon: 37 + icon·2 + daytime	`TP/WX/Icon (+0.5 = day), App 27`
`55 - 215`	TalonCast temperature −40..120 (your units)	`TP/WX/T (temp + 100), Valid = 2, App 27`
`216 - 217`	Units °F / °C	`TP/WX/Units, App 27`
`218 - 228`	PDA size 60..160 %	`TP/SF/Size`
`229 - 231`	Custom theme 5-7 (2.13)	`TP/SF/Theme`
`232 - 255`	Spare for extensions	—

App messages repeat every 0.1 s for four ticks, so switches show up within about a quarter of a second; other messages take 0.25 s slots. The receiver re-applies the last value every couple of frames with drivers that have `localOnly = false`.

> **NOTE**
> Your city name, forecast, ticker, text and pet are never sent. Other players' TalonCast shows a single 'Live from this PDA' page.

Beam: data over contacts

Avatars can't read each other's parameters, so Beam carries a message between two Talon Pilots on contacts. The sender's own client steps through the message on `TP/Net` (0.4 s per symbol) and raises the synced bit `TP/Beam`, which tells every other client to stop decoding `TP/Net` as app and theme messages.

Where	What happens
Sender, own client	Net Send: preamble (128), start (2), the 15 characters, end (3). Bit 7 toggles on every symbol: it is the clock.
Sender's avatar, on every other client	Net Receive copies TP/Net into `TP/NetF`; a Display-layer state plays a 1D blend tree with 256 children that switches 8 contact senders (`TalonBeam0-7`) on the PDA to that symbol's bits.
Receiver, own client	8 local receivers (others only) feel the senders when the PDAs are within about a hand's width. Each toggle of bit 7 latches bits 0-6 through one decode state (128 transitions, checked only on that frame), then stores the character. Start clears the inbox, end opens Beam and chirps.

Cost: 1 synced bit, 16 contacts, about 30 parameters and no layers. When nobody is beaming, the only per-frame work is one transition check.

08

The TalonSync Link Protocol

A clocked serial bus made of avatar parameters. Protocol version 3.

TalonSync and the PDA talk only through avatar parameters over OSC. The PC drives the bus; the PDA's TalonSync Bus layer answers. VRChat only reports parameters that change, so before a read the PC writes sentinel values to the data lines; anything still at its sentinel is known to be unchanged.

Signals

Parameter	Type	Direction	Meaning
`TP/Bus/Addr`	Int	PC → PDA	Port address 0-254
`TP/Bus/Clk`	Bool	PC → PDA	Raised to start a transfer
`TP/Bus/Ack`	Int	PDA → PC	1 done, 2 unknown address, 0 released
`TP/Bus/Len, V, N`	Int, Float, Int	both	Length / small value; float value; integer value
`TP/Bus/C0-14`	Int	both	Characters (one line)
`TP/Bus/B0-23`	Bool	both	Ink row bits (older PDAs with a Note Pad only)
`TP/Sync/Req`	Bool	PDA → PC	The PDA's TalonSync button is waiting
`TP/Sync/State`	Int	PC → PDA	0 idle, 1 PC wants to sync, 2 transferring, 3 done, 4 failed
`TP/Sync/Stage, Prog`	Float	PC → PDA	Stage / 8 and progress 0..1 for the PDA's screen

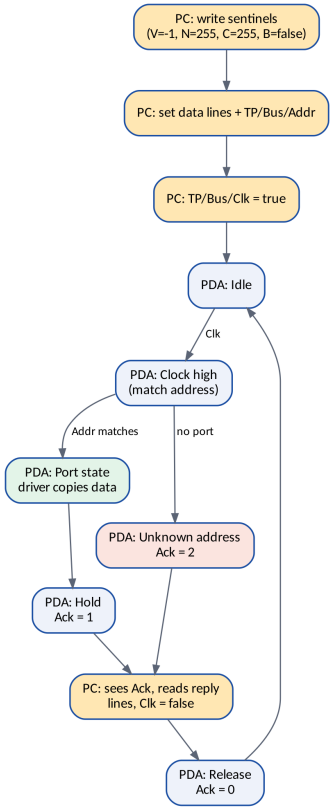


Figure 8-1. One bus transfer.

Address map

Address	Operation
`0 - 13`	PC Link value k (Len, raw 0-255; 255 = not available): CPU, GPU, CPU °C, GPU °C, RAM, headset, left, right, heart bpm, timer minutes, timer fraction, timer state, unread
`14, 15`	PC Link message lines (C0-14)
`16`	Alert: the PDA chirps
`17`	Clear the PC Link messages
`18, 19`	Moon & Stars latitude (Len - 90) and longitude (Len / 255 × 360 - 180)
`0 - 63 (protocol 2)`	Note Pad ink rows, on PDAs before 2.10
`64 - 111`	Write text slot s (C0-14, Len)
`112 - 159`	Read text slot s
`160 - 199`	Read value k (V for floats, N for ints/bools)
`200`	Ping: N = protocol version
`201`	Reload display copies (after writing prefs)
`202`	Clear the Note Pad (PDAs before 2.10)
`203, 204`	Now Playing line 0 / 1
`205`	Now Playing position (Len / 255)
`206`	Now Playing clear
`207 - 246`	Write value k (Len; floats Len / 255)
`247`	Open app Len (and power on)
`248`	Weather now (C0-14)
`249`	Forecast (C0-9)
`250`	City name
`251`	Jazz mode (Len / 2)
`252 - 254`	Ticker lines 0-2

Value table (addresses 160 + k and 207 + k)

k 0-19: the first 20 preferences in table order (Hour, Minute, Clock24, Bright, Theme, Wall, FX, Sound, Vol, TapBeep, GameSnd, Btn1-4, Saver, SaverTime, AutoOff, BootLock, ClockSrc); 20-25: app settings (3D model, look, spin; Hypno speed, style; Scope mode); 26-33: task flags; 34: power; 35-39: pet (food, joy, energy, age, asleep).

> **RULE**
> The value table is exactly 40 entries and both sides depend on its order. New preferences are appended to the prefs table but kept off the bus (TalonPilotSettings.BusPrefs = 20).

A TalonSync session

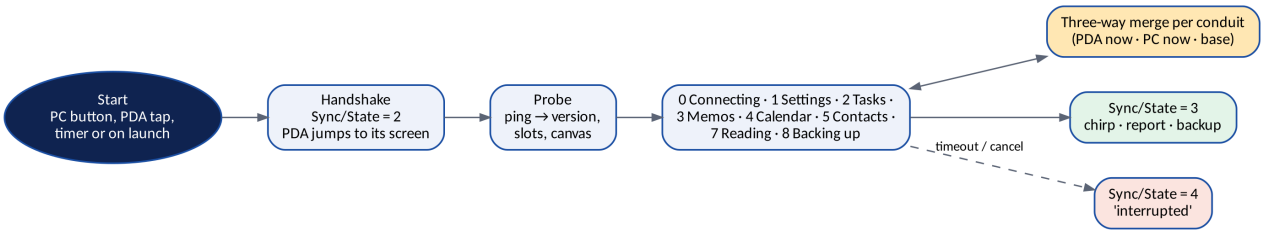


Figure 8-2. Sync session stages.

09

TalonSync Software Architecture

A Python service with a web front end.

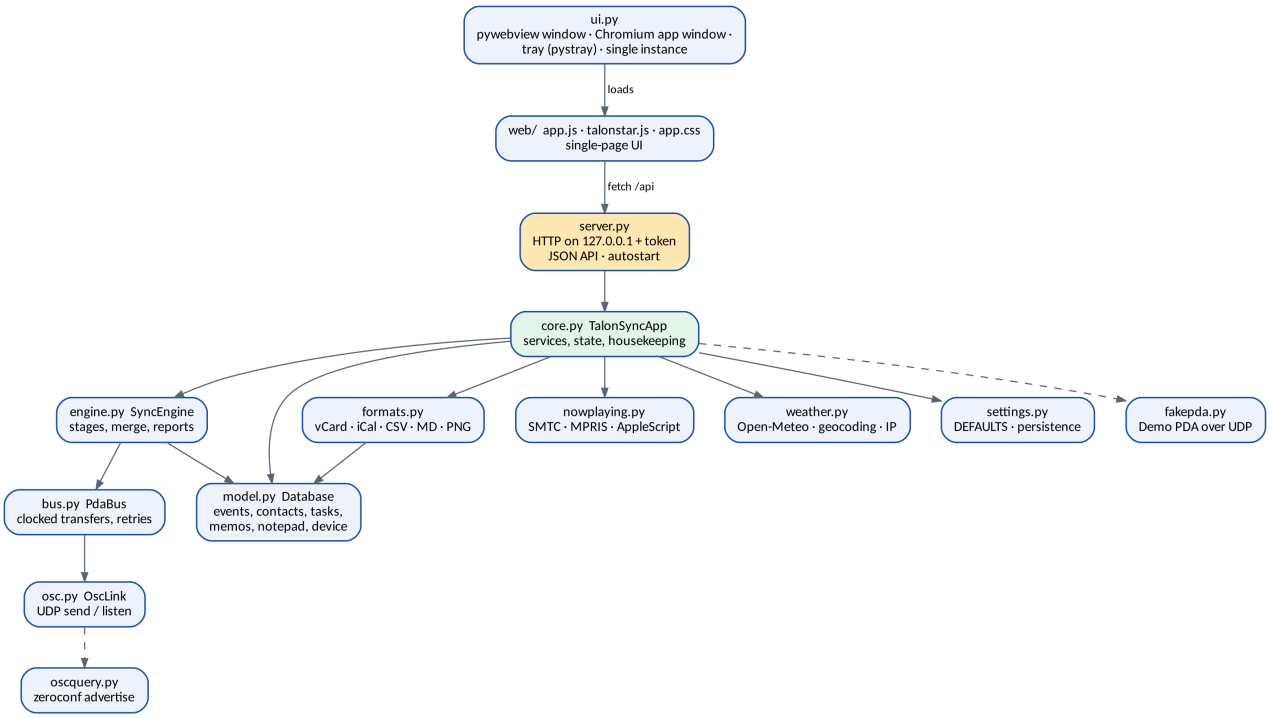


Figure 9-1. TalonSync modules.

Module	Responsibility
`talonsync_main.py`	Entry point and command-line options: --minimized, --browser, --port, --data, --demo, --headless.
`core.py`	TalonSyncApp: owns settings, database, OSC link, bus, sync engine, Now Playing, weather and housekeeping; builds the UI state.
`engine.py`	SyncEngine: runs a session through its stages and conduits, three-way merges, reports and backups.
`bus.py`	PdaBus: clocked transfers with sentinels, timeouts and retries; high-level operations.
`protocol.py`	Parameter names, addresses, stages, slot map and the value table: must match the Unity package.
`osc.py / oscquery.py`	UDP OSC send and receive; OSCQuery advertising over zeroconf.
`model.py / formats.py`	Records and the file formats (vCard, iCalendar, CSV, Markdown, PNG, JSON).
`nowplaying.py / weather.py`	Media sources per OS; Open-Meteo forecasts, geocoding and IP location.
`server.py / ui.py / web/`	Local HTTP API with a per-run token; window, tray and autostart; the single-page interface.
`fakepda.py`	Demo PDA: a pretend Talon Pilot that speaks the bus over UDP.

Scripting the PDA from Python

```
from talonsync.osc import OscLink
from talonsync.bus import PdaBus, Timing

link = OscLink(send_port=9000, listen_port=9001)       # VRChat's default ports
link.start()
bus = PdaBus(link, Timing(ack_timeout=1.0))

print("protocol", bus.ping())                        # 2
bus.open_app(27)                                     # open TalonCast
bus.write_text(0, "Bake lightmaps")                  # task slot 0
bus.reload()                                         # refresh the display copies
bus.now_playing("Midnight Glide", "The Night Owls", position=0.25)
bus.ticker("Doors open at 9 pm in the Aerie!")
```

PdaBus method	Address(es)	Notes
`ping() → int`	`200`	Protocol version
`probe() → PdaInfo`	`200, 160+`	Version, text slots, canvas size
`read_text(slot) / write_text(slot, text)`	`112+s / 64+s`	15 ASCII characters, right-aligned on the PDA
`read_ink_row(r, cols) / write_ink_row(r, bits) / clear_ink()`	`32+r / 0+r / 202`	Note Pad rows (PDAs before 2.10)
`read_value(k) / write_value(k, v)`	`160+k / 207+k`	See the value table
`reload()`	`201`	After writing prefs
`now_playing(title, artist, position) / now_position(p) / now_clear()`	`203-206`	
`open_app(app)`	`247`	Also powers the PDA on
`weather(w) / ticker(text) / jazz_mode(mode)`	`248-254`	TalonCast
`pc_value(k, raw) / pc_message(title, text) / pc_alert() / pc_clear()`	`0-17`	PC Link (protocol 3)
`sky_location(lat, lon)`	`18, 19`	Moon & Stars
`set_state(s) / set_stage(stage, progress)`	`TP/Sync/*`	Drive the PDA's TalonSync screen

> **TIP**
> Only one program should drive the bus at a time. Quit TalonSync (or use its Demo PDA) before running your own scripts against VRChat.

10

Developer SDK Guide

Extending TalonOS: the toolchain and step-by-step recipes.

The toolchain

Generated files must not be edited by hand. The generators ship in `Assets/TalonPilot/SDK/Generators` and write into the package (override the location with the `TALONPILOT_ROOT` environment variable). They need Python 3.8 or newer and no extra packages.

Generator	Writes	Source of truth for
`gen_tables.py`	`Editor/TalonPilotSettings.cs`, `Shaders/Core/Talon_Tables.cginc`	Preferences, categories, About text, keyboard layout
`gen_registers.py`	`Shaders/Core/Talon_Registers.cginc (+ properties block)`	Every register and the index-search functions
`gen_sky.py`	`Shaders/Core/Talon_SkyData.cginc`	The Moon & Stars catalog: bright stars and constellation lines
`gen_core.py`	`Talon_Registers.cginc, Talon_Strings.cginc, TalonOS_Core.shader`	Compiled-in strings; assembles the shader from TalonOS_Core.shader.template

```
cd Assets/TalonPilot/SDK/Generators
python gen_tables.py        # after changing preferences or the keyboard
python gen_core.py          # after changing registers, strings or the template (also runs gen_registers)
```

Then let Unity reimport, and run the Install Wizard's **Update** on your avatar so the animator is rebuilt from the C# generators.

Recipe: add a preference

- Append a `pref(...)` line at the end of the list in gen_tables.py (never insert: saved TalonSync data uses table order). Max 7 per category (2.23: a page with more than 4 drops its name and hint rows); 29 total, one ROM row per name in PREF_LABEL_ROWS2 (add a free row there for more).
- Kinds: 1 labels (needs value labels; ROM rows 960-999 are nearly full), 2 number, 3 bar, 4 app name.
- Run gen_tables.py then gen_core.py. The Prefs app, the animator steppers, the display mirror and the expression parameter appear automatically.
- Read it in the shader with `Talon_S(TALON_PREF_MYKEY)`.

Recipe: add a compiled-in string

```
# gen_core.py, strings list
("MYAPP_HELLO", "Hello from my app"),
# shader
col = lerp(col, c.fg, Talon_StrText(c, float2(20, 60), TALON_S_MYAPP_HELLO, 1.0));
```

Recipe: add an app

Free app ids are 14-17 (the shared channel carries apps 0-31; 28-31 became Owl Oracle, Moon & Stars, PC Link and Beam in 2.11, 21-22 TalonPop and Snow Globe in 2.15, 19 the Aquarium in 2.16, 18 the Terminal in 2.18). Ids 9 and 13 belonged to the retired Note Pad and Hoot Pop: don't reuse them, because saved button settings may still point at them. The example adds app 17 on launcher page 2, after the Terminal.

1. Draw it. Add the function to Talon_Apps2.cginc and route it inside the TP_PLAY block of TalonApps2_Draw:

```
#if TP_PLAY
    ...
    if (Is(app, 17)) { col = TalonApp_MyApp(c, col); return true; }
#endif
```

2. Title. Add a string ("T_MYAPP") in gen_core.py and return it from TalonApps2_Title for app 17.

3. Ownership. In TalonOS_Core.shader.template, include 17 in Talon_IsExtensionApp and in the Play branch of Talon_OwnerPart:

```
if ((a > 22.5 && a < 28.5) || Is(a, 31) || Is(a, 17)) return 1.0;
```

4. Launcher. Add app 17 to a category in LauncherCategories (TalonPilotAnimatorApps.cs) and LAUNCHER_CATEGORIES (gen_registers.py), in the same position (8 slots per category), and draw an icon in TalonIcon_Proc. The launcher table is generated from these lists for both the animator and the material (2.14).

```
new[] { 27, 25, 24, 29, 17 },   // Sky & Tools, C#
[27, 25, 24, 29, 17],            # Sky & Tools, gen_registers.py
```

5. Install it. Add 17 to AllApps and a name in AppName (TalonPilotAnimatorApps.cs).

6. Input. In a Build*Input method, use the generator helpers with an app condition:

```
var here = L(IEq(P("App"), 17)).ToList();
Stepper("MyApp level", P("My/Level"), 0, 9, 1, false, "Right", "Left", here);
On("Ok", Action("MyApp go", Set(P("My/Go"), 1)), here);
```

7. Registers. Declare them in gen_registers.py and feed them from a gated Direct tree, so they cost nothing outside your app:

```
DirectTreeLayer("MyApp Registers", 17, new List<(string, string)> { (P("My/Level"), "_MyLevel") });
```

8. Regenerate, Update the avatar, and test in Gesture Manager or Av3Emulator.

> **TIP**
> Saved parameters you add must be listed in the wizard's expression parameter section (EPar) and should be local (synced = false). Put new menu items under the existing Talon Pilot submenu.

Optional apps (2.14)

Apps 20 and 23-31 can be left out in the wizard. `TalonPilotAnimator.Without` holds the left-out ids and `HasApp(id)` gates everything an app builds: its parameters, its Display state, its input states, its bus ports, its layers (Hoot Pet's two), its Net states (TalonCast's weather, Beam's "I'm beaming" conditions via `NotBeaming()`) and its objects (Beam's contacts, the Oracle's shake PhysBone, the jazz players). The launcher compacts: `LauncherTable(without)` fills page 1 with the first 15 apps and adds More only when there are more. The C# table and the material table come from the same function. Bus addresses of a left-out app answer NAK; TalonSync treats that as "not on this PDA". Keep the 40-entry value table unchanged: value ports whose parameter doesn't exist are simply not built.

When you add an app, decide whether it is optional. If it is, add it to `OptionalApps` and to the wizard's `AppCosts`, gate its Build* calls with `HasApp`, and run the harness's "without" configurations: no condition or driver may reference a missing parameter.

Power-on self-test (2.17)

Every boot line is a real check, and the boot lasts only as long as the slowest one (VRChat restoring saved memory, TalonSync answering; at most 1.5 s). The Power layer runs a short chain of states that exist only while booting; nothing runs after boot.

Check	How it is measured
ROM	The shader sums one byte from each of the 1024 ROM rows (column (7 r + 3) mod 32) and compares it with the checksum the wizard stores in the ROM itself (row 0, bytes 29-31). Only the pixels of the ROM line run the sum.
RAM	POST writes the pattern 165 into `TP/Boot/Res`; the Display layer copies it to `_BootRes` and the shader checks it arrived intact: driver, parameter memory, blend tree, material, GPU.
Saved memory	The saved int `TP/Boot/NV` is 165 once a PDA has booted: 165 restored, 0 first boot (then set), anything else bad (then reset).
PC link	TalonSync answers power-on by setting `TP/PC/Hello` (its version, 187 = 1.8.7) over OSC, three times within the window.
Input, digitizer	VRMode and TrackingType from VRChat; any touch line pressed during POST is reported as held.
CPU clock, display, RTC, audio, battery, apps	Shader: 1 / unity_DeltaTime, the real pixel size of the screen, VRChat's time globals, the AudioLink texture, the TalonSync battery value, the launcher table.

All results travel in one float: `TP/Boot/Res` = (165 + 256 × flags) / 65536, exact in a float. Flags: 1 NVRAM ok, 2 blank (1+2 bad), 4 TalonSync, 8 touch held, 16 VR, 32 full body, 64 finished. The Display layer's "Show POST" state is the first Any State transition and may re-enter itself, so it holds the layer while Res > 0 without adding a condition to any other transition; after boot it costs one failed check a frame. Cost: 3 parameters (+ VRChat's TrackingType), 1 animated value, 0 layers.

Terminal: the PC as the CPU (2.18)

App 18 is a 30 x 8 character screen for Talon Enhanced BASIC, which runs in TalonSync (talonsync/basic.py, talonsync/terminal.py). TalonSync writes the screen over OSC into 128 local parameters `TP/Term/L{w}_{k}`: row r is text-window lines 2r (columns 0-14) and 2r+1 (15-29), two characters per parameter (c1 × 128 + c2). The Display state copies them straight into the 16-line text-window registers the other text apps use, so the terminal adds one register (`_TermCur`, row × 32 + column, −1 = no PC) and no animated values while closed. Only changed lines are sent, at most about 12 screens a second. BASIC is not started on its own: at power-on, avatar load, TalonSync start or when `TP/App` becomes 18, TalonSync shows a start menu (1 Talon Enhanced BASIC, 2 local terminal with built-in commands); BYE or SYSTEM returns from BASIC to the local terminal. The start menu has three choices since 1.10.2: 1 Program Library (talonsync/library.py), 2 BASIC, 3 local terminal. The CPU is experimental and opt in (TalonSync setting `exp_terminal`, off by default); while it is off TalonSync ignores commands 40-44 and keeps `TP/Term/Cur` at −1.

Typing reuses the QWERTY keyboard (ESlot 98). Done copies the edit buffer into `TP/Bus/C0-14` and `TP/Bus/Len` and sets `TP/PC/Cmd` = 40; 41/42 scroll back and forward, 43 breaks, 44 repeats the last line. A full 15-letter line is joined with the next send. `TP/Term/Beep` plays the chirp (BASIC's BEEP). POST sets `TP/Term/Cur` to −1 and TalonSync redraws when it answers power-on. Cost when installed: about 130 local parameters, 0 layers, two Display states and one Beep-layer check; left out: nothing.

Line graphics (2.18.2). Codes 5-31 are not font glyphs on the Terminal: Talon_TermGfx draws them as shapes that fill the whole 10 x 34 character cell (a letter only uses 20 of the 34 units), so lines meet across rows and columns. 5/6/7 shades (25/50/75 %), 8 upper half, 9 lower half, 11 left half, 12 right half, 13 ball, 14 / and 15 \ diagonals, 16 full block, and 17-31 box pieces = 16 + arms (1 up, 2 down, 4 left, 8 right): 19 vertical, 28 horizontal, 26 top-left corner, 31 cross. 10 is the new line and never reaches the PDA. TalonSync maps the Unicode box and block characters to these codes, keeps a cursor so the screen is addressable (LOCATE), and ORs the arms when a line is drawn over a line, so crossings join by themselves. BASIC: LOCATE r,c; BOX r,c,h,w; LINE r1,c1,r2,c2. A line's end on empty ground is drawn full length; on another line it becomes a T.

TalonOS Enhanced (2.19, TalonSync 1.11). TalonSync draws a desktop (talonsync/enhanced.py) as 30 x 8 cells of a character and a 5-bit style, packed into the same 128 terminal registers: v = c1 × 128 + c2 + 16384 × (s1 + 32 × s2), still exact in a float. `TP/Term/Cur` ≥ 1000 switches the PDA to desktop mode (≥ 2000: the start-up screen). Talon_TermGui turns cells into 10 × 52-unit tiles: each style has a background, an ink and a bevel family (window, button, pressed, field, taskbar, tray, tooltip, start button); bevels are drawn where a neighbour belongs to another family, so TalonSync only says what each cell is. Anchor styles (20/21 icon, 28 owl, 29 small owl) draw a 30-unit picture over their cell and its neighbours: 'A'-'P' the icon atlas, 'f' folder, 'p' prompt, 'r' bin. Five cell reads per pixel share one inlined fetch; the Terminal runs in the Aqua part since 2.21. Input: Up/Down 41/42, Left 43 (back), Right 44 (Start menu), Center 45 (Enter, desktop mode only), typed lines 40 as shortcuts. Taps: the 32 + 24 touch-grid contacts (`TP/In/IR0-31`, `TP/In/IC0-23`) are local expression parameters, so VRChat reports them over OSC; TalonSync averages the active rows and columns 90 ms after a touch starts and hit-tests the cell. In desktop mode the animator leaves taps alone.

TalonOS Enhanced 1.13 (Talon Pilot 2.20, TalonSync 1.13). A window manager (enhanced.py: Desktop, App, an immediate-mode UI toolkit of labels, buttons, fields, check boxes, radio buttons, list boxes and progress bars) with system apps (tos_apps.py) and games (tos_games.py). Each frame every open window redraws its cells and registers its focusable controls in drawing order: Up/Down move the focus (a focused list box takes them first), Center presses, typed lines go to the focused field or the app. The touch grid is coarser than the screen (24 contacts across 30 cells), so a tap that lands on window background snaps to the nearest control within 0.65 cell. `TP/Term/Cur` 1000 = Classic, 1500 = Skyglass (second palette TG_SKY, glossy title bars, a sky-and-hill desktop with label shadows, a green Start button). Buttons (families 2 and 3) are always one row tall and inset 5 units top and bottom, so stacked buttons (keypads, game boards) stay apart; a window cell frames itself only toward the outside, never toward the controls inside it. The Terminal moved to the Aqua part in 2.21 (about 3k instructions). Settings and scores: talonos.json in TalonSync's data folder; notes in Documents, deleted files in Recycled.

The TalonOS kernel (TalonSync 1.14). tos_kernel.py keeps a process table (PID, kind system / basic / plugin, CPU time, faults); every call into an app (draw, key, typed, tick, taps) goes through Kernel.run, so an exception closes only that app, logs it to C:\Temp\errors.log and shows a dialog. Services: an event bus (fs.changed, settings.changed, clipboard.changed, process.started / ended, app.*), a registry with a namespace per app (in talonos.json), a clipboard, notification balloons and recent documents. TalonFS (tos_fs.py) maps C:\Documents, Programs, Apps, Recycled and Temp onto folders in the data folder, and A:\ onto the built-in BASIC programs (read only); paths are case-insensitive, '..' stops at the root, links are refused and every real path is checked to stay inside its mount. Limits: 32-letter names, 1 MB files, 64 MB on C:, 1,000 entries a folder. Apps reach all of this through self.api (fs with a per-app current folder, settings, ui dialogs, events, clipboard, notify, launch, open). BASIC gains file I/O (OPEN #, PRINT #, INPUT #, CLOSE, EOF, EXISTS) and window statements (WINDOW, TEXT, BUTTON, FIELD, CHECK, BAR, WAIT, NOTIFY, MSGBOX, FIELD$, CHECKED); a program with WINDOW runs as a BasicApp process on its own thread. Python plugins (C:\Apps*.py, class Main from talonsync.tos_sdk) are off by default and need consent per file hash. Guide: docs/TalonOS_SDK.md.

Clocks without layers (2.15)

Since 2.17.6 the Snow Globe is a real 3D scene: the glass ball is solved exactly (ray/sphere, Fresnel, refraction into water), then the scene is ray-marched along the short chord inside the ball with bounding tests per object, a 5-tap loop for the normal and ambient occlusion (so the scene code compiles only twice), fewer steps when far away, and snow on five depth layers. It costs less than the 2.15 flat version. It needs a harder shake than the Owl Oracle (GlobeShakeThreshold 0.55).

Snow Globe needs time without adding a layer: its Display state plays a clip that animates the register `_GlobeT`. A shake flips the Gen bool, and the app has two identical Display states (A and B) gated on it, so the flip re-enters a state and restarts the clip.

TalonPop (2.15.2) keeps its game clock in the Idle layer, which only runs the screensaver timer and rests while TalonPop is open: a loop of two 0.4 s tick states. Each tick adds 1 to the round (`TP/SB/T`) and to each star's age (`TP/SB/A{i}`),

then checks, in order: leaving, round over (150 ticks or no lives), bursts (age past the star's life), next tick. The tick clip animates `_SBPhase` 0-1 so stars grow smoothly between ticks. The Input layer only handles taps (pop: points by age band), so a game can never hold up the buttons. Taps raise TP/Poke as usual; the Idle layer ignores Poke while a round is running. Whether a score beats the best needs no comparison of two parameters: `TP/SB/D` starts at −Best and every point adds to it, so a new best is D > 0.

The Aquarium (2.16) uses the same trick for its feeding clock (`_AquaT`, flipped by `TP/Aqua/Gen`). Everything else, the fish paths, turning, tail beats and swimming to the food, is worked out in the shader from that one clock, so the animator does no work while you watch. Its loops use [loop] rather than [unroll], so eight detailed fish cost the instructions of one.

Stereo depth (2.21). The Aquarium and Moon & Stars are drawn from the viewer's eye. The vertex shader passes the screen point in object space; the fragment shader turns the camera position (`_WorldSpaceCameraPos`, which is per eye in VR) into screen pixels using the uv derivatives (`TA_Eye` in Talon_Eye.cginc): x and y on the screen and the distance in front of it. A point at depth D behind the screen then lands at E + (W − E) · d / (d + D), and each layer of the tank (back water, rock, corals, sand, fish, seagrass) is found by intersecting this pixel's ray with it, so each eye sees its own view. The bowfront glass bulges 110 px out of the screen at its centre and fish may swim into it; the moon is ray-traced as a sphere 70 px in front of the screen. Previews use `_AquaEye` instead. Fish are one parametric body (an airfoil-like profile with a lowered snout, gill cover and lips) with fins made of rays and webbing (`TA_Rays`); the gill covers and the mouth move with a breathing cycle. Fish highlights are written as HDR values (up to about 3) so world bloom picks them up.

Depth on every screen (2.22). Every program now works out the eye (`TA_Eye`) and the whole picture is drawn 6 px behind the glass, with the case edge shading it; the launcher's wallpaper sits at 30 px with the icons' soft shadows on it; TalonCast's backdrop at 40 px; Hoot Pet's sky at 160 px and its branch at 30 px; Scope's screen at 24 px; Hypnospiral's tunnel up to 240 px at its heart; Snow Globe's room at 300-700 px with the ball 24 px in front of the glass; Owl Oracle's ball 46 px in front, ray-traced, with the answer on a plane 34 px inside its window. Prefs › Effects › 3D depth (`TP/S/Depth`, pref 23) sets `g_depth` = 0 and a fixed eye, so everything draws flat.

Eye candy (2.23). After the scene (before the LCD grid) the fragment pipeline adds the **touch ripple** (two rings from `_TapX`/`_TapY` for 1.1 s after a press, 4 px in front of the glass) and the **idle glow** (the screen edge breathing in mint while `_TapT` > 20). After the brightness it adds the **glass glints**: the reflected ray of the real eye (`g_eyeR`, kept even when 3D depth is off) against a long light, so the streaks move as you do. The launcher's icons take the **holo foil** colour (`TalonFoil()`) when `_Foil` is 1. `TE_Motes()` (Talon_Eye.cginc) draws drifting motes on a plane at any depth, one per hashed cell: marine snow in the Aquarium (two planes, hidden behind fish and rock) and fireflies around the Owl Oracle. Each has its Prefs › Effects switch (prefs 25-28: Glints, Ripples, Glow, Motes). The wallpapers (Aurora, Morning mist) are compiled into the OS program only, the one that draws the launcher.

The case foil (2.23) is a second renderer, *Body/Foil*, with the body mesh and one material (`TalonPilot/Foil`), so only the shell submesh draws: additive, ZWrite off, a small depth offset. Thin-film colour from the viewing angle, a fine diagonal grating that fades with distance, a highlight towards the main light, all scaled by the world's ambient and main light. `_FoilOn` comes from TP/Foil in the Display layer (and in its powered-off state); at 0 the vertex shader collapses the mesh.

GPS (2.21) is a glass cockpit drawn from the PDA's own transform: attitude from the screen's axes, speed from `_Speed`, height and position from the object-to-world matrix, and HOME as the world origin. The moving map's terrain is noise sampled at the world point under each pixel, so it scrolls and turns with you without any parameters.

Recipe: add a bus command

- Pick a free address (255 is the only one left: extend by repurposing a range only with a protocol version bump).
- Add a `Port(addr, name, drivers...)` in BuildBus (TalonPilotAnimatorBus.cs); drivers copy from the data lines.
- Add the constant and a PdaBus method in talonsync/protocol.py and bus.py, and teach fakepda.py the address.
- Bump PROTOCOL_VERSION on both sides when an existing meaning changes.

Compatibility rules

- Never reorder or remove preferences, text slots or value-table entries: saved avatar data and TalonSync backups depend on positions.
- Keep TP/App, TP/On and TP/Net names and types; other players' clients rely on them.
- Keep the text encoding (code / 127 per saved character, two characters per display register).
- Record new animator paths before the animator is built: anything that moves objects (anchors, placements) must run first.
- Keep synced bits at 10 (11 with shared jazz). Add shared information as TP/Net message ranges (232-255 are spare).

Performance budget

Resource	Budget	Current
Transition checks per frame (local, on)	< 400	≈ 215
Transition checks per frame (off / remote)	< 200	≈ 120
Direct-tree children per frame	< 350 in any app	61 launcher · 197 Tasks · 302 Memos
Animated values (bindings)	< 900	≈ 280
Animator parameters	< 1,300	≈ 1,200
Layers added to FX	< 25	18
Expression parameters	< 1,000	≈ 900
Synced bits	≤ 11	10-11
Largest shader program	< 20,000 DXIL	≈ 18,700

RULE
On your own avatar, VRChat runs the animator again for your mirror and shadow copies, and community benchmarks put local parameters at about 1.5 ms per 1,000. Parameters and layers cost every frame even while the PDA is off, so they are the budget that matters most. Note Pad (about 200 parameters) and Hoot Pop (about 105 parameters and a layer) were retired in 2.10 for this reason.

Testing checklist

- Shader compiles with no warnings in all three keyword variants and with no keyword.
- Every app renders the same pixels in its own program as in the all-in-one program.
- Input: each new transition is reachable from the Pressed hub; hold-and-release does not double-fire.
- Remote: with IsLocal false, parked layers stay parked and TP/Net decodes correctly.
- TalonSync: ping, text round-trip, values and any new port against the Demo PDA, then in VRChat.

11

Appendices

A. Package layout

```
Assets/TalonPilot/
  Editor/                C# generators: wizard + Control Center, animator (CPU), ROM, mesh, TalonSync push
  Shaders/TalonOS.shader classic shader (v1.6) for minimal installs
  Shaders/Core/          TalonOS Core shader and modules (see chapter 6)
  Textures/              font atlas, icon atlas
  Audio/                 tap, chirp, pop, TalonCast jazz
  SDK/Generators/        Python generators for tables, registers, strings and the shader
  Documentation/         Handbook, Quick Start Guide, this document
  Generated/<avatar>/    created per avatar: materials, meshes, ROM, menus, backups
TalonSync/
  talonsync/             Python package (see chapter 9) and web/ interface
  tests/                 fake-PDA sync, file formats, real-animator bridge tests
  Build TalonSync.bat    Windows build · Run TalonSync.sh / .command for Linux and macOS
```

B. Glossary

Term	Meaning
Register	A material property the shader reads; written by animator blend trees.
Mirror	A Float copy of an Int parameter (TP/SF/*), because blend trees only take Floats.
Gated layer	A layer whose Direct tree only plays while its app is open, locally.
Parked	A layer state with one exit, used for other players and while powered off.
Hub	The Input layer's Pressed state, which holds all app transitions.
Program / part	One of the three keyword variants of the TalonOS Core shader.
Slot	A saved 15-character text line; also the shader pattern of drawing one primitive per pixel.
Conduit	One kind of information TalonSync synchronizes (tasks, memos …).
Sentinel	A value the PC writes before a bus read, so unchanged replies are detectable.

Talon Pilot is an independent product and is not affiliated with or endorsed by VRChat. VRChat is a trademark of its owner.